Chapter 16: Object Relationships

ENG3091

Khiem Nguyen

Contents

1 Object relationships **2**
- 1.1 Relationships between objects . . . 2
- 1.2 Four relationships, from strongest to weakest . . . 2

2 Object composition **2**
- 2.1 Building objects from parts . . . 2
- 2.2 Composition . . . 3
- 2.3 Composition, explained with a heart . . . 3
- 2.4 A familiar composition: `Fraction` . . . 3
- 2.5 Implementing compositions . . . 4
- 2.6 A creature's location: `Point2D` . . . 4
- 2.7 A creature has a location . . . 5
- 2.8 Playing the creature game . . . 6

3 Aggregation **6**
- 3.1 Aggregation . . . 6
- 3.2 Shared, not owned . . . 7
- 3.3 Aggregation: two examples . . . 7
- 3.4 Implementing aggregations . . . 7
- 3.5 A department and its teachers . . . 8
- 3.6 The teachers outlive the department . . . 9
- 3.7 Composition versus aggregation . . . 9

4 Association **9**
- 4.1 Association . . . 9
- 4.2 Doctors and patients . . . 10
- 4.3 Implementing associations . . . 10
- 4.4 The other side: `Doctor` . . . 11
- 4.5 A bidirectional association in action . . . 12

5 Dependencies **12**
- 5.1 Dependencies . . . 12
- 5.2 Dependency or association? . . . 13

6 Wrapping up **13**
- 6.1 The relationships at a glance . . . 13
- 6.2 Summary . . . 14
- 6.3 Keep learning . . . 14

1 Object relationships

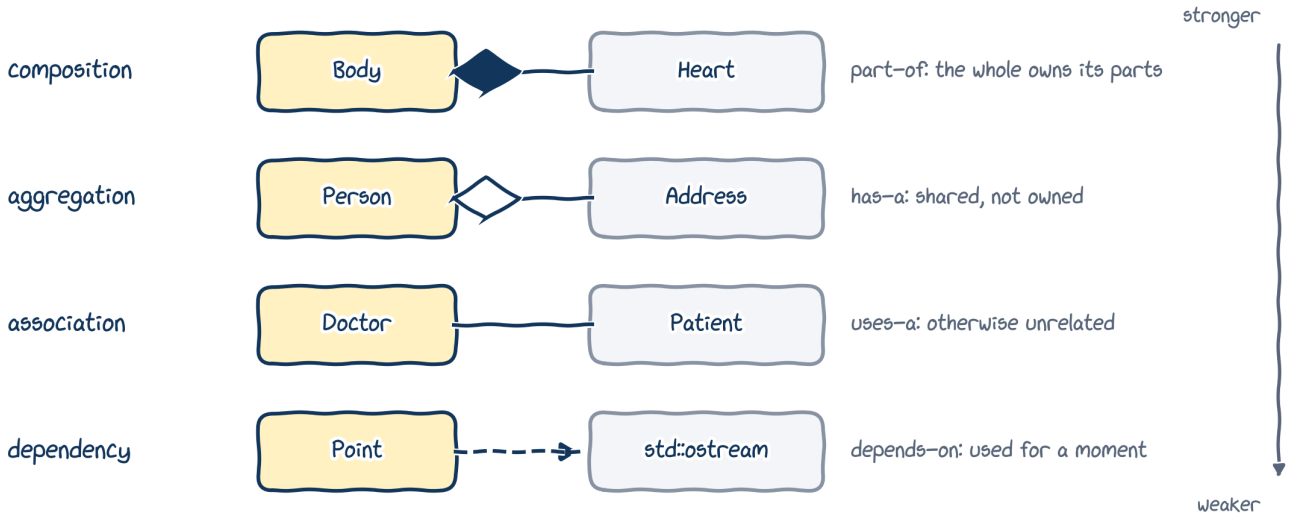
1.1 Relationships between objects

Life is full of objects that relate to each other, and we have everyday words for the kinds of relationship:

- a square **is a** shape; a lecturer **is a** university person (inheritance, Chapter 14);
- a car **has a** steering wheel; a person **has a** heart;
- a programmer **uses a** keyboard;
- a flower **depends on** a bee for pollination;
- a student is a **member of** a class; your brain is **part of** you.

Today: the relationships *other than* "is a", and how to write them as C++ classes. In Chapter 17, we'll draw them as UML diagrams.

1.2 Four relationships, from strongest to weakest



The little diamonds and arrows are a preview of UML notation (Chapter 17).

2 Object composition

2.1 Building objects from parts

Building a complex object from simpler ones is called **object composition**. You've been doing it since Chapter 10: every class with data members is built from parts.

- A computer is built from a CPU, a motherboard, some memory…
- A person is built from a head, a body, arms, legs…

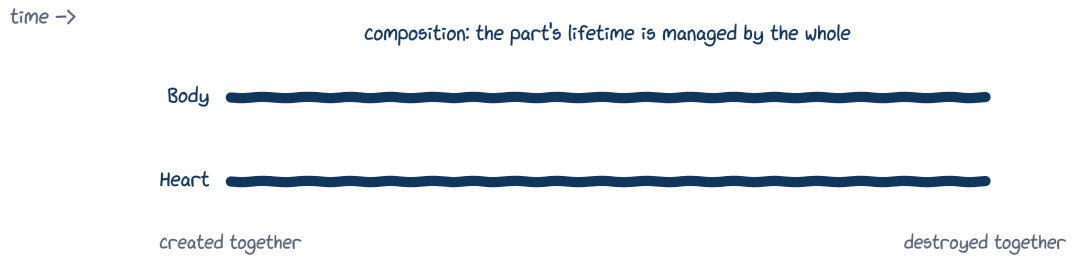
Object composition models **"has a"** relationships: a computer *has a* CPU; a person *has a* heart. It comes in two kinds: **composition** and **aggregation**.

> ⚠ Watch the terminology
>
> *Object composition* is the general idea; *composition* and *aggregation* are its two subtypes. Don't confuse "object composition" with plain "composition".

2.2 Composition

In a **composition**, a part and its whole have this relationship:

1. the part is **part of** the whole;
2. the part can belong to **only one** whole at a time;
3. the part's **existence is managed** by the whole;
4. the part **doesn't know** about the whole.



2.3 Composition, explained with a heart

1. **Part of:** a heart is part of a person's body.
2. **One whole at a time:** a heart in one person's body can't be in someone else's at the same time.
3. **Managed by the whole:** when the body is created, so is the heart; when the body is destroyed, so is the heart.
4. **Doesn't know:** the heart works away, unaware that it's part of a larger structure. The body knows about the heart, but not the other way round: a **unidirectional** relationship.

2.4 A familiar composition: `Fraction`

Listing 1 `fraction_composition.cpp`

```cpp
class Fraction {
  private:
    int numerator{};   // numerator and denominator are PARTS of a Fraction:
    int denominator{}; // created with it, destroyed with it, never shared

  public:
    Fraction(int numerator = 0, int denominator = 1)
        : numerator{numerator}, denominator{denominator} {}
};

int main() {
    Fraction f{3, 4};
    return 0;
}
```

- `numerator` and `denominator` are part of the `Fraction`, and belong to only one `Fraction`.
- They're created and destroyed with it.
- They don't know they're part of a `Fraction`: they just hold integers.

Obvious? Be patient: it's not always this simple.

2.5 Implementing compositions

- Usually, the parts are simply **ordinary data members**: their lifetimes are automatically bound to the object's.
- If the parts need dynamic allocation (Chapter 09), use **pointer members**, and the class itself must do all the memory management: allocate in the constructor, free in the destructor (RAII, Chapter 12). The user of the class should never have to.

2.6 A creature's location: `Point2D`

Listing 2 `composition/point2d.h`

```
#ifndef POINT2D_H
#define POINT2D_H

#include <iostream>

class Point2D {
  private:
    int x{};
    int y{};

  public:
    Point2D(int x = 0, int y = 0) : x{x}, y{y} {}

    void set(int new_x, int new_y) {
        x = new_x;
        y = new_y;
    }

    friend std::ostream& operator<<(std::ostream& out, const Point2D& p) {
        return out << '(' << p.x << ", " << p.y << ')';
    }
};

#endif
```

2.7 A creature has a location

Listing 3 composition/creature.h

```
#ifndef CREATURE_H
#define CREATURE_H

#include "point2d.h"
#include <iostream>
#include <string>

class Creature {
  private:
    std::string name{};
    Point2D location{}; // composition: a part of every Creature

  public:
    Creature(const std::string& name, const Point2D& location)
        : name{name}, location{location} {}

    void move_to(int x, int y) { location.set(x, y); }

    friend std::ostream& operator<<(std::ostream& out, const Creature& c) {
        return out << c.name << " is at " << c.location;
    }
};

#endif
```

2.8 Playing the creature game

Listing 4 `composition/creature_game.cpp`

```cpp
#include "creature.h"
#include <iostream>
#include <string>

int main() {
    std::cout << "Enter a name for your creature: ";
    std::string name{};
    std::cin >> name;
    Creature creature{name, Point2D{4, 7}};

    while (true) {
        std::cout << creature << '\n';

        std::cout << "New x and y (or -1 -1 to quit): ";
        int x{};
        int y{};
        std::cin >> x >> y;
        if (x == -1) {
            break;
        }
        creature.move_to(x, y);
    }
    return 0;
} // creature is destroyed, and its Point2D with it
```

```
$ ./creature_game
Enter a name for your creature: Gromit
Gromit is at (4, 7)
New x and y (or -1 -1 to quit): 1 2
Gromit is at (1, 2)
New x and y (or -1 -1 to quit): -1 -1
```

Every `Creature` **has a** `Point2D` location: a part of it, created and destroyed with it, and unaware of the creature it belongs to. A composition.

3 Aggregation

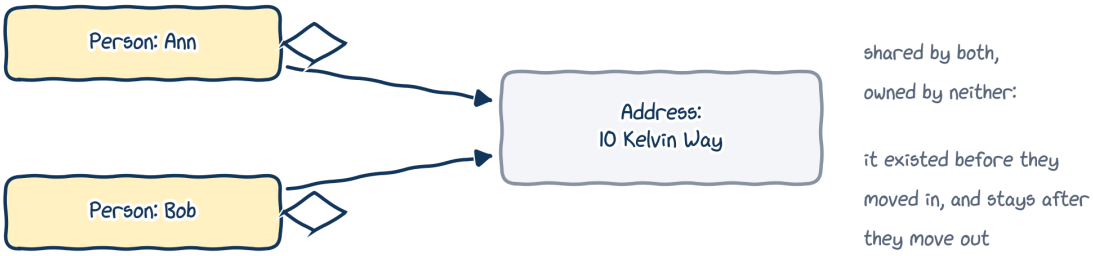
3.1 Aggregation

In an **aggregation**, a part and its whole have this relationship:

1. the part is **part of** the whole;
2. the part can (if desired) belong to **more than one** whole at a time;
3. the part's **existence is not managed** by the whole;
4. the part **doesn't know** about the whole.

Still a part–whole, unidirectional relationship. But the whole doesn't create or destroy its parts, and parts may be shared.

3.2 Shared, not owned



Compare with the composition timeline: there, the heart lived and died with the body. Here, the address has a life of its own.

3.3 Aggregation: two examples

A person and their home address

- Every person has an address.
- An address can belong to several people at once.
- The address isn't managed by the person: it existed before they moved in, and still exists after they move out.
- A person knows their address; an address doesn't know who lives there.

A department and its teachers

- A department has teachers.
- A teacher could work for more than one department.
- Closing a department doesn't destroy its teachers: they existed before it, and go on existing after it.
- The department knows its teachers; a teacher needn't know the department.

3.4 Implementing aggregations

Compositions and aggregations look almost the same in code: the parts are still data members. The difference is mostly about **ownership**:

- in an aggregation, the members are usually **references or pointers** to objects created **outside** the class;
- when the aggregate is destroyed, those references or pointers are destroyed, but the objects they refer to are not.

A `std::vector` can't hold plain references (Chapter 07: a reference must be bound when it's created, and can't be reseated). **`std::reference_wrapper<T>`**, from `<functional>`, is a reference that *can* be stored in a vector.

3.5 A department and its teachers

Listing 5 `school.h`

```
#ifndef SCHOOL_H
#define SCHOOL_H

#include <functional> // std::reference_wrapper
#include <iostream>
#include <string>
#include <vector>
struct Teacher { // passive data, so a struct (Chapter 10)
    std::string name{};
};

class Department {
  private:
    std::vector<std::reference_wrapper<const Teacher>> teachers{};

  public:
    void add(const Teacher& teacher) { teachers.push_back(teacher); }

    void print() const {
        std::cout << "Department staff:";
        for (const Teacher& teacher : teachers) {
            std::cout << ' ' << teacher.name;
        }
        std::cout << '\n';
    }
};
#endif
```

`Department` stores `std::reference_wrapper<const Teacher>`s: references to teachers who live elsewhere. That's what makes it an aggregation.

`Teacher` is just data, so it's a `struct` (Chapter 10).

3.6 The teachers outlive the department

Listing 6 `teacher_department.cpp`

```cpp
#include "school.h"
#include <iostream>

int main() {
    Teacher tony{"Tony"}; // created OUTSIDE any department
    Teacher natasha{"Natasha"};
    {
        Department department{};
        department.add(tony);
        department.add(natasha);
        department.print();
    } // the department is destroyed here...

    std::cout << tony.name << " and " << natasha.name
              << " still exist\n"; // ...but its teachers are not
    return 0;
}
```

```
Department staff: Tony Natasha
Tony and Natasha still exist
```

The department never created its teachers, so it doesn't destroy them either.

3.7 Composition versus aggregation

	Composition	Aggregation
Members are usually	ordinary data members (or pointers it manages itself)	references or pointers to outside objects
Creates and destroys its parts?	yes	no
Parts shared with other objects?	no	possibly

- A single class can mix both: responsible for some parts, but not others.
- Aggregations are more error-prone: whoever creates the parts must make sure they live long enough (no dangling references, Chapter 07).

> Best practice
>
> Implement the simplest relationship that meets the needs of your program, not the one that seems right in real life.

4 Association

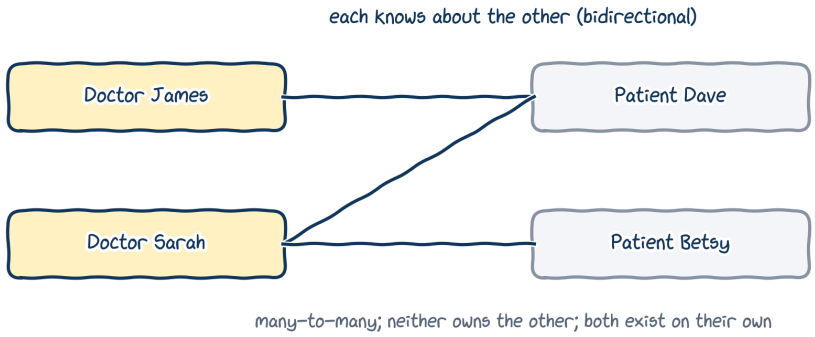
4.1 Association

In an **association**, two objects have this relationship:

1. the associated object is **otherwise unrelated** to the object: not a part of it;
2. the associated object can belong to **more than one** object at a time;
3. the associated object's **existence is not managed** by the object;
4. the associated object **may or may not know** about the object.

Unlike composition and aggregation, there's no part–whole relationship. And unlike aggregation, an association may be **bidirectional**: both objects know about each other.

4.2 Doctors and patients



- A doctor clearly has a relationship with their patients, but a patient isn't *part of* a doctor.
- A doctor sees many patients, and a patient may see many doctors.
- Neither object's lifetime is tied to the other's.

4.3 Implementing associations

Associations are usually implemented with **pointers** (or references) to the associated objects.

Listing 7 `patient.h`

```
#ifndef PATIENT_H
#define PATIENT_H

#include <string>
#include <vector>

class Doctor; // forward declaration: Patient refers to Doctor

class Patient {
  private:
    std::string name{};
    std::vector<const Doctor*> doctors{}; // a patient knows their doctors

  public:
    Patient(const std::string& name) : name{name} {}
    const std::string& get_name() const { return name; }
    const std::vector<const Doctor*>& get_doctors() const { return doctors; }
    void add_doctor(const Doctor& doctor) { doctors.push_back(&doctor); }
};

#endif
```

`class Doctor;` is a forward declaration: a `Patient` only stores pointers to doctors, so the compiler just needs to know the name.

4.4 The other side: `Doctor`

Listing 8 `doctor.h`

```h
#ifndef DOCTOR_H
#define DOCTOR_H

#include "patient.h"
#include <string>
#include <vector>

class Doctor {
  private:
    std::string name{};
    std::vector<const Patient*> patients{}; // a doctor knows their patients

  public:
    Doctor(const std::string& name) : name{name} {}
    const std::string& get_name() const { return name; }
    const std::vector<const Patient*>& get_patients() const { return patients; }

    void add_patient(Patient& patient) {
        patients.push_back(&patient);
        patient.add_doctor(*this); // keep both sides of the link in sync
    }
};

#endif
```

4.5 A bidirectional association in action

Listing 9 `doctor_patient.cpp`

```cpp
#include "doctor.h"
#include <iostream>

int main() {
    Patient dave{"Dave"}; // patients and doctors exist independently
    Patient betsy{"Betsy"};
    Doctor james{"James"};
    Doctor sarah{"Sarah"};

    james.add_patient(dave);
    sarah.add_patient(dave); // Dave sees two doctors...
    sarah.add_patient(betsy);

    std::cout << "Sarah sees:";
    for (const Patient* patient : sarah.get_patients()) {
        std::cout << ' ' << patient->get_name(); // ...Sarah sees two patients
    }
    std::cout << "\nDave sees:";
    for (const Doctor* doctor : dave.get_doctors()) {
        std::cout << ' ' << doctor->get_name();
    }
    std::cout << '\n';
    return 0;
}
```

```
Sarah sees: Dave Betsy
Dave sees: James Sarah
```

`add_patient()` updates **both** sides, so the links never get out of step. A bidirectional association needs that care; when you can, prefer a unidirectional one.

5 Dependencies

5.1 Dependencies

A **dependency** occurs when one object uses another object's functionality to accomplish a specific task, **without keeping** it.

Listing 10 `dependency_example.cpp`

```cpp
#include <iostream>

class Point {
  private:
    double x{}, y{}, z{};

  public:
    Point(double x = 0.0, double y = 0.0, double z = 0.0) : x{x}, y{y}, z{z} {}

    // Point depends on std::ostream: it USES one, but doesn't keep one
    friend std::ostream& operator<<(std::ostream& out, const Point& p) {
        return out << "Point(" << p.x << ", " << p.y << ", " << p.z << ')';
    }
};

int main() {
    Point p{42.0, 24.0, 42.24};
    std::cout << p << '\n';
    return 0;
}
```

```
Point(42, 24, 42.24)
```

`Point` uses a `std::ostream` to print itself, but doesn't keep one.

5.2 Dependency or association?

- In an **association**, one class keeps a **link** to the other as a **member**. You can always ask a `Doctor` who its patients are.
- In a **dependency**, the object used is typically **not a member**. It's created when needed (like opening a file to write to), or passed in as a parameter (like `std::ostream` in `operator<<`).

6 Wrapping up

6.1 The relationships at a glance

Property	Composition	Aggregation	Association	Dependency
Relationship	whole/part	whole/part	otherwise unrelated	otherwise unrelated
Can be shared	no	yes	yes	yes
Lifetime managed by the other	yes	no	no	no
Direction	unidirectional	unidirectional	either	unidirectional
In words	"part of"	"has a"	"uses a"	"depends on"

6.2 Summary

- **Object composition** builds complex objects from parts: **composition** (the whole owns its parts) and **aggregation** (the whole only refers to them).
- Compositions use ordinary data members, which live and die with the object. Aggregations usually use references or pointers to objects that live elsewhere.
- An **association** links otherwise unrelated objects, often in both directions, usually through pointers.
- A **dependency** uses another object for a moment, without keeping it.
- Implement the simplest relationship that does the job.

6.3 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com, Chapter 23:

1. Object relationships
2. Composition and aggregation
3. Association, including reflexive and indirect associations
4. Dependencies
5. Container classes and `std::initializer_list`

Next up, Chapter 17: drawing all of these as UML class diagrams.